Проектирование БД. Нормализация

Agenda (План занятия)

Жизненный цикл базы данных

Зачем нужно проектирование? Проблемы плохого дизайна.

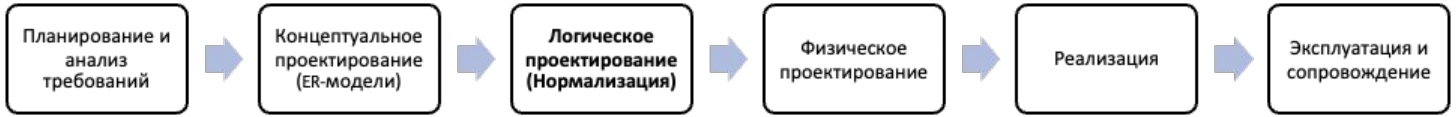
Нормализация: цели и преимущества

Подробный разбор нормальных форм:

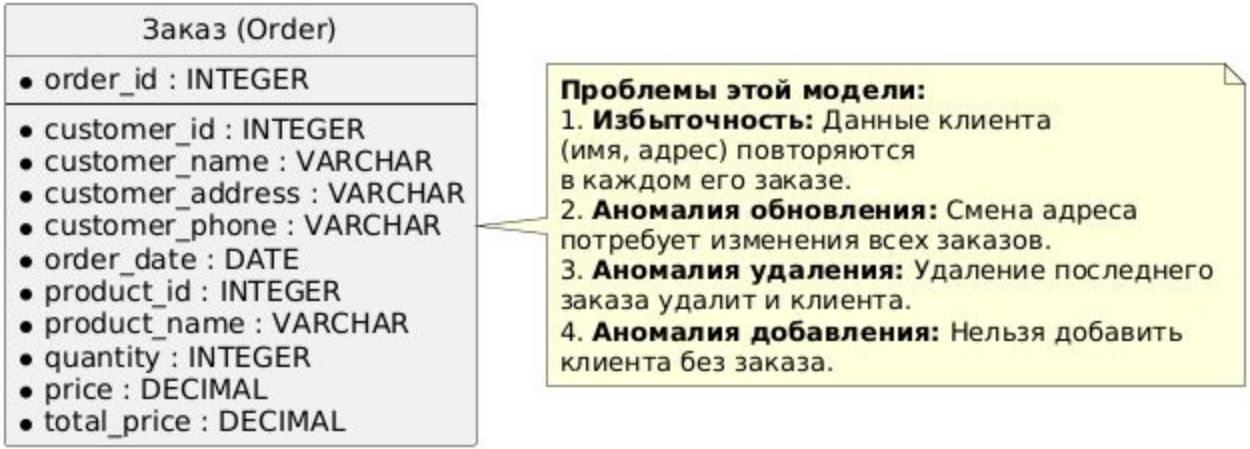
- Первая нормальная форма (1NF)
- Вторая нормальная форма (2NF)
- Третья нормальная форма (3NF)

Денормализация: когда и зачем?

Жизненный цикл



Почему важно проектирование?



Нормализация - это...

Процесс организации данных в базе для уменьшения избыточности и улучшения целостности. Достигается путем декомпозиции (разделения) таблиц на более мелкие и связывания их через ключи.

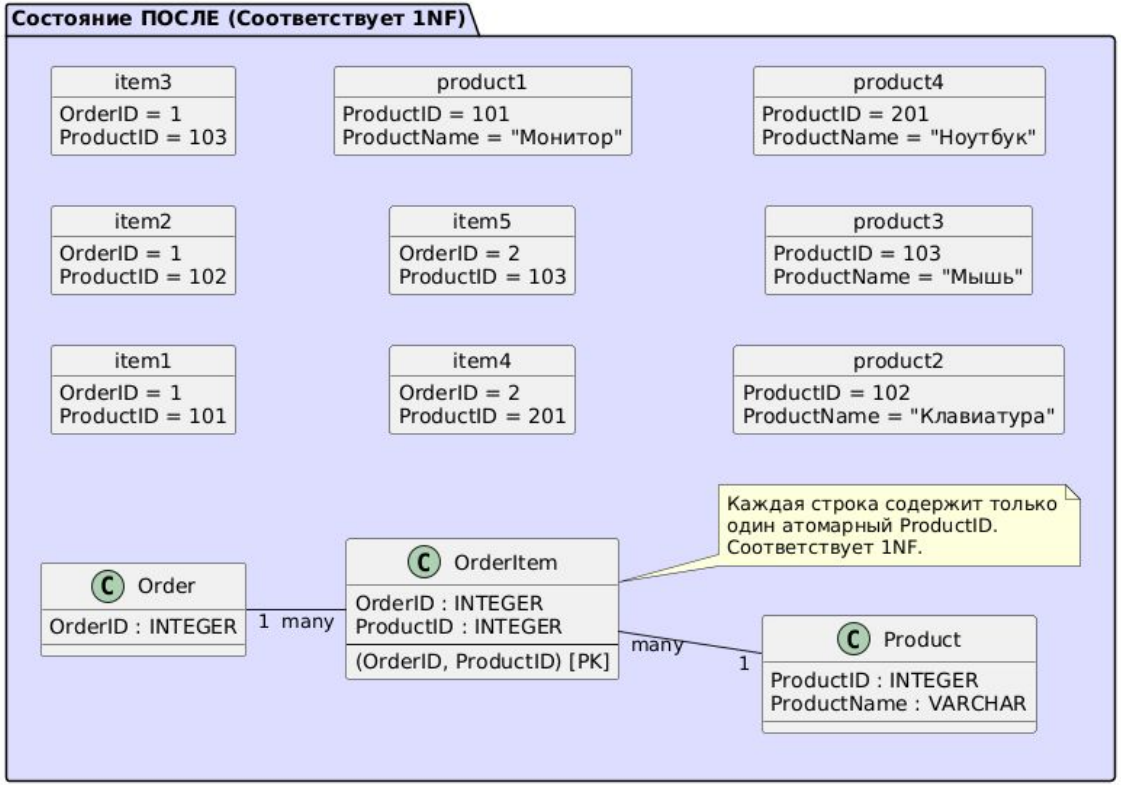
Цели:

- Исключить избыточность данных.
- Предотвратить аномалии (обновления, удаления, добавления).
- Обеспечить логическую, непротиворечивую структуру данных.

Первая нормальная форма (1NF)

Ячейки таблицы должны содержать только атомарные (неделимые) значения. Все атрибуты содержат только скалярные (простые) значения.

Пример приведения к 1NF



Функциональные зависимости и ключи

Функциональная зависимость (ФЗ): X -> Y. Значение X однозначно определяет значение Y.

Пример: OrderID -> OrderDate (по ID заказа можно однозначно найти его дату).

Потенциальный ключ - атрибут(-ы), которые уникально идентифицируют запись.

Первичный ключ (PK) - выбранный потенциальный ключ.

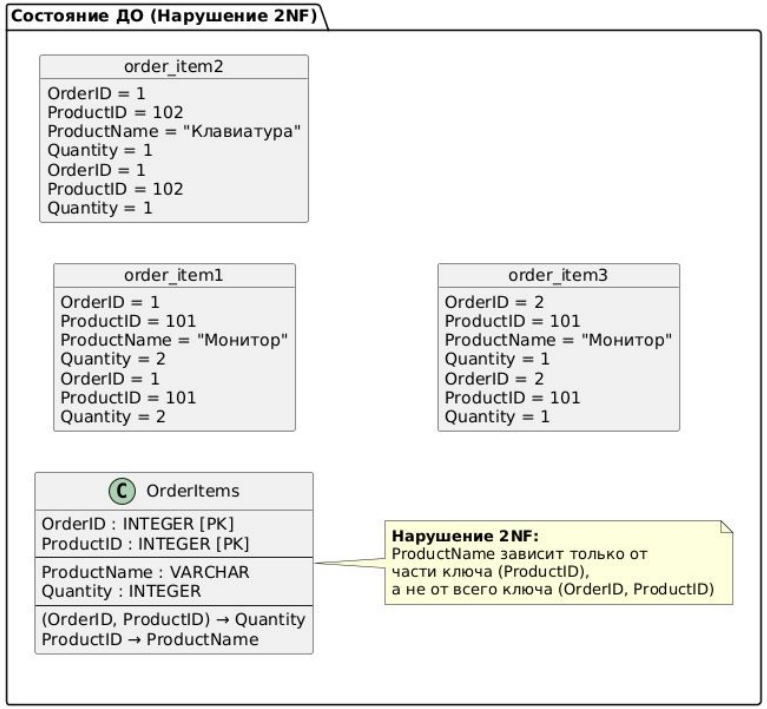
Неключевой атрибут - все атрибуты, не входящие в PK.

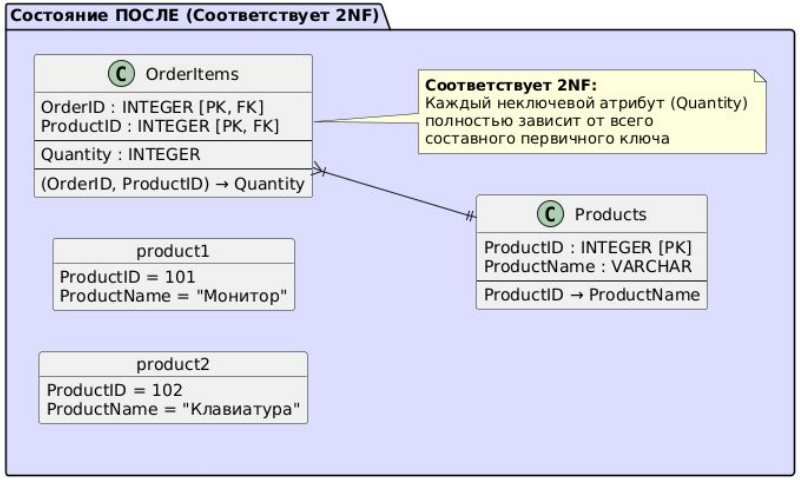
Вторая нормальная форма (2NF)

Начальные условия: Таблица должна находиться в 1NF.

Каждый неключевой атрибут должен полностью функционально зависеть от всего первичного ключа.

Проблема: Если PK составной, и некоторые атрибуты зависят только от части PK.


Состояние ДО (Нарушение 2NF)
order_item2
OrderID = 1
ProductID = 102
ProductName = "Клавиатура"
Quantity = 1
OrderID = 1
ProductID = 102
Quantity = 1
order_item1
OrderID = 1
ProductID = 101
ProductName = "Монитор"
Quantity = 2
OrderID = 1
ProductID = 101
Quantity = 2
order_item3
OrderID = 2
ProductID = 101
ProductName = "Монитор"
Quantity = 1
OrderID = 2
ProductID = 101
Quantity = 1
OrderItems
OrderID : INTEGER [PK]
ProductID : INTEGER [PK]
ProductName : VARCHAR
Quantity : INTEGER
(OrderID, ProductID) → Quantity
ProductID → ProductName
Нарушение 2NF:
ProductName зависит только от
части ключа (ProductID),
а не от всего ключа (OrderID, ProductID)

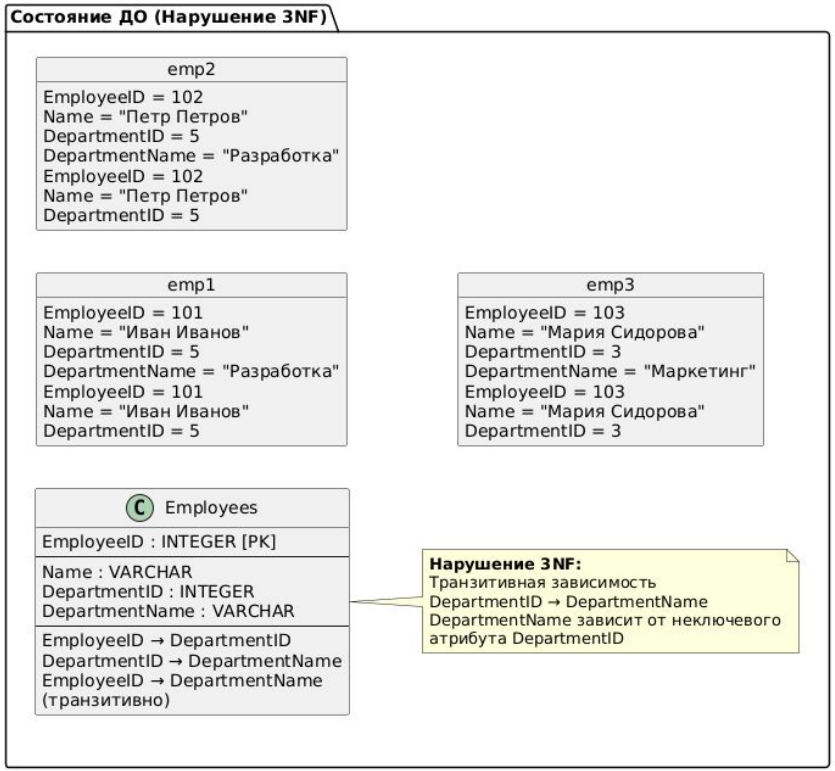

Состояние ПОСЛЕ (Соответствует 2NF)
OrderItems
OrderID : INTEGER [PK, FK]
ProductID : INTEGER [PK, FK]
Quantity : INTEGER
(OrderID, ProductID) → Quantity
Соответствует 2NF:
Каждый неключевой атрибут (Quantity)
полностью зависит от всего
составного первичного ключа
Products
ProductID : INTEGER [PK]
ProductName : VARCHAR
ProductID → ProductName
product1
ProductID = 101
ProductName = "Монитор"
product2
ProductID = 102
ProductName = "Клавиатура"

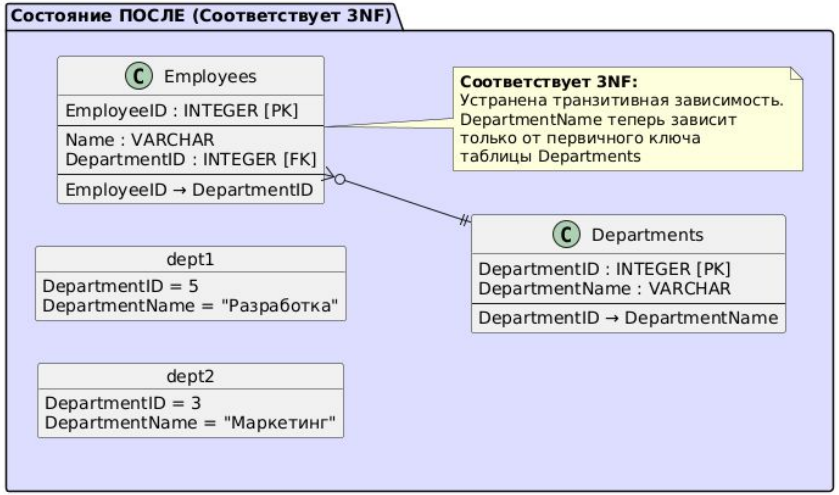
Третья нормальная форма (3NF)

Начальные условия: Таблица должна находиться в 2NF.

Отсутствие транзитивных зависимостей. Никакой неключевой атрибут не должен зависеть от другого неключевого атрибута.

Все неключевые атрибуты должны зависеть только от первичного ключа.


Состояние ДО (Нарушение 3NF)
emp2
EmployeeID = 102
Name = "Петр Петров"
DepartmentID = 5
DepartmentName = "Разработка"
EmployeeID = 102
Name = "Петр Петров"
DepartmentID = 5
emp1
EmployeeID = 101
Name = "Иван Иванов"
DepartmentID = 5
DepartmentName = "Разработка"
EmployeeID = 101
Name = "Иван Иванов"
DepartmentID = 5
emp3
EmployeeID = 103
Name = "Мария Сидорова"
DepartmentID = 3
DepartmentName = "Маркетинг"
EmployeeID = 103
Name = "Мария Сидорова"
DepartmentID = 3
Employees
EmployeeID : INTEGER [PK]
Name : VARCHAR
DepartmentID : INTEGER
DepartmentName : VARCHAR
EmployeeID → DepartmentID
DepartmentID → DepartmentName
EmployeeID → DepartmentName
(транзитивно)
Нарушение 3NF:
Транзитивная зависимость
DepartmentID → DepartmentName
DepartmentName зависит от неключевого
атрибута DepartmentID


Состояние ПОСЛЕ (Соответствует 3NF)
Employees
EmployeeID : INTEGER [PK]
Name : VARCHAR
DepartmentID : INTEGER [FK]
EmployeeID → DepartmentID
Соответствует 3NF:
Устранена транзитивная зависимость.
DepartmentName теперь зависит
только от первичного ключа
таблицы Departments
Departments
DepartmentID : INTEGER [PK]
DepartmentName : VARCHAR
DepartmentID → DepartmentName
dept1
DepartmentID = 5
DepartmentName = "Разработка"
dept2
DepartmentID = 3
DepartmentName = "Маркетинг"

Денормализация

Преднамеренное отступление от правил нормализации для повышения производительности, с целью ускорения выполнение запросов (чаще всего операций чтения - SELECT) за счет:

- Сведения JOIN'ов к минимуму.
- Предварительного расчета агрегатов.
- Введения контролируемой избыточности.

Важно! Денормализация следует после нормализации, а не вместо нее. Это осознанный компромисс между производительностью и целостностью.

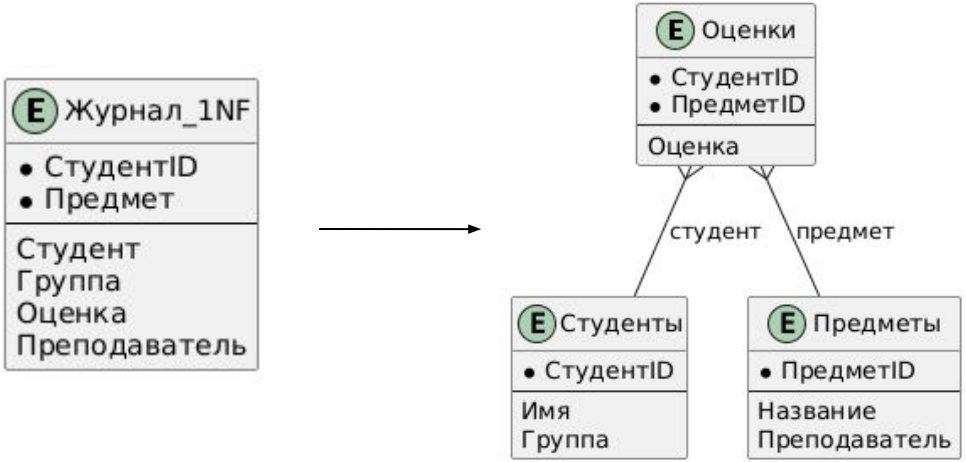
0NF

Журнал_0NF
• СтудентID
Студент
Группа
Предметы_и_оценки
Преподаватели

0NF — > 1NF

Разбиваем составные поля на отдельные строки. Делаем каждую запись атомарной.

1NF — > 2NF

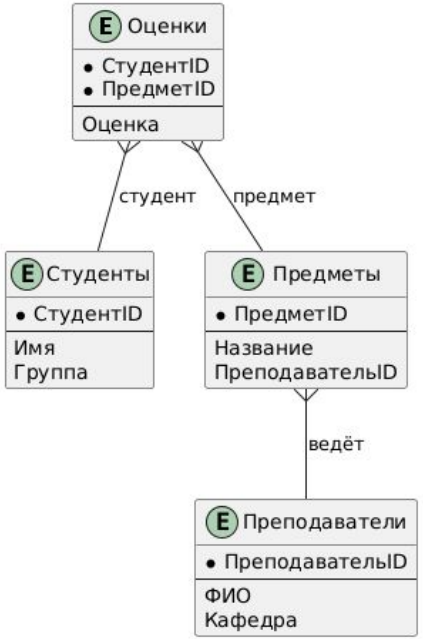


Устраняем частичные зависимости.

Данные студентов → в отдельную таблицу *Студенты*
Данные предметов и преподавателей → в *Предметы*

НО: если преподаватель зависит от кафедры, а кафедра — от предмета, то может быть транзитивная зависимость → проверяем 3NF.

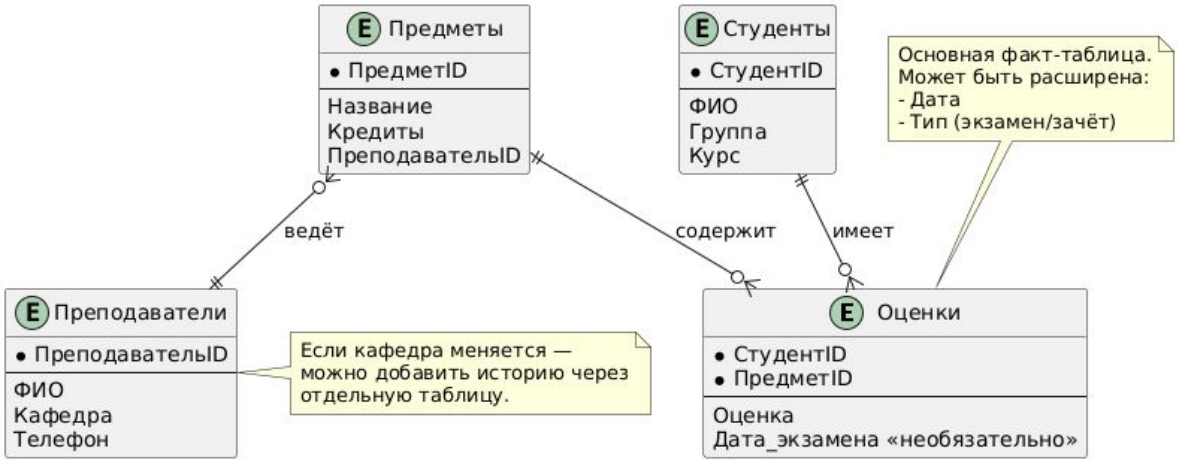
2NF — > 3NF



Устраняем транзитивные зависимости.

Выносим преподавателей в отдельную таблицу.

ИТОГ



- Нет дублирования данных.
- Обеспечена целостность (например, нельзя поставить оценку несуществующему студенту).
- Гибкая — легко добавить новые предметы, студентов, изменить преподавателя.

Для аналитики/отчётов может потребоваться денормализация — например, добавить в “Оценки” поля **`ФИО_студента`**, **`Название_предмета`**, **`ФИО_преподавателя`** для ускорения SELECT-запросов.

Проблема циклических зависимостей

Что такое циклическая связь таблиц

Цикл в связях возникает, когда внешние ключи (foreign keys) образуют замкнутый круг.
Например:

- Таблица **A** ссылается на **B**
- Таблица **B** ссылается на **C**
- Таблица **C** ссылается обратно на **A**

В итоге, при вставке/удалении данных возникает ситуация "курица и яйцо" — чтобы создать запись в одной таблице, уже нужны записи в другой, которая, в свою очередь, требует первой.

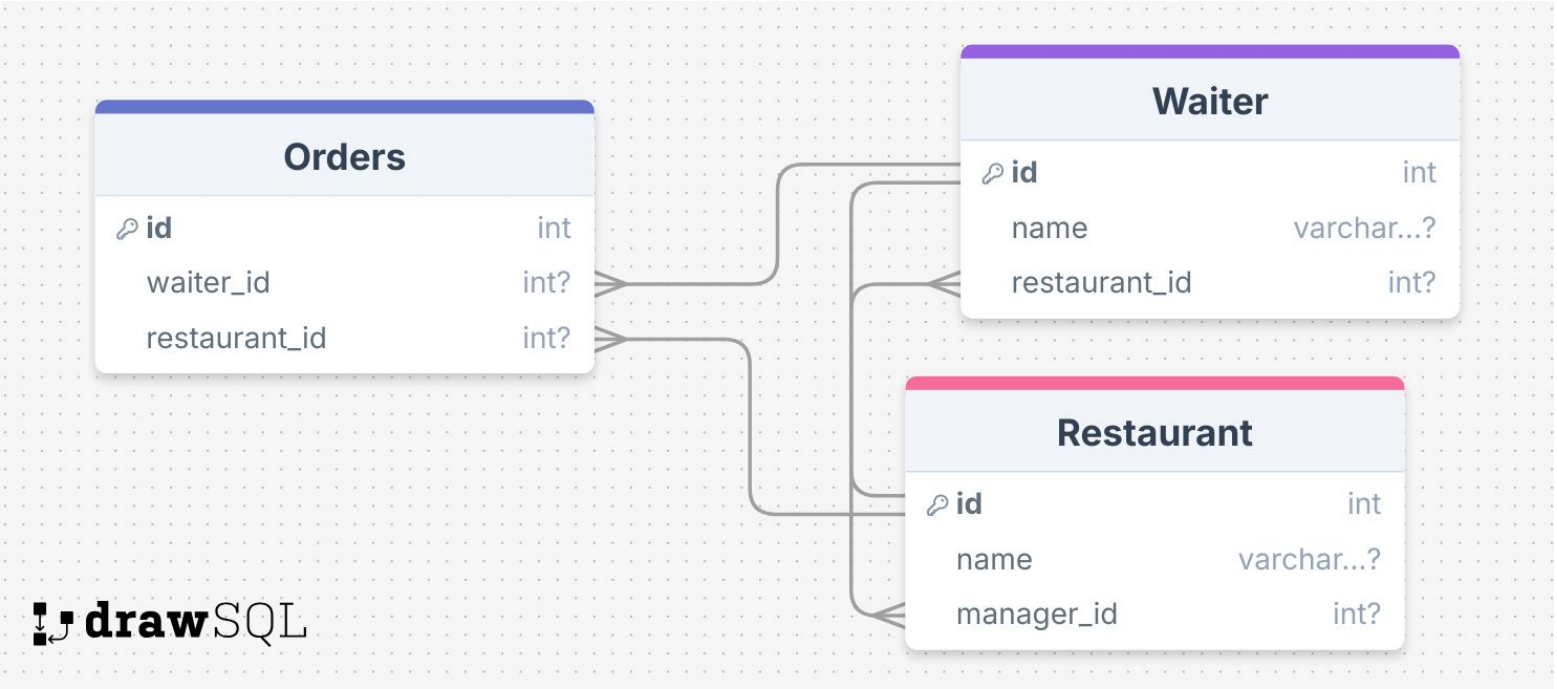
В чем проблема?

Проблема целостности данных
невозможно вставить строку, пока не существует связанной строки, а та строка тоже требует первой.

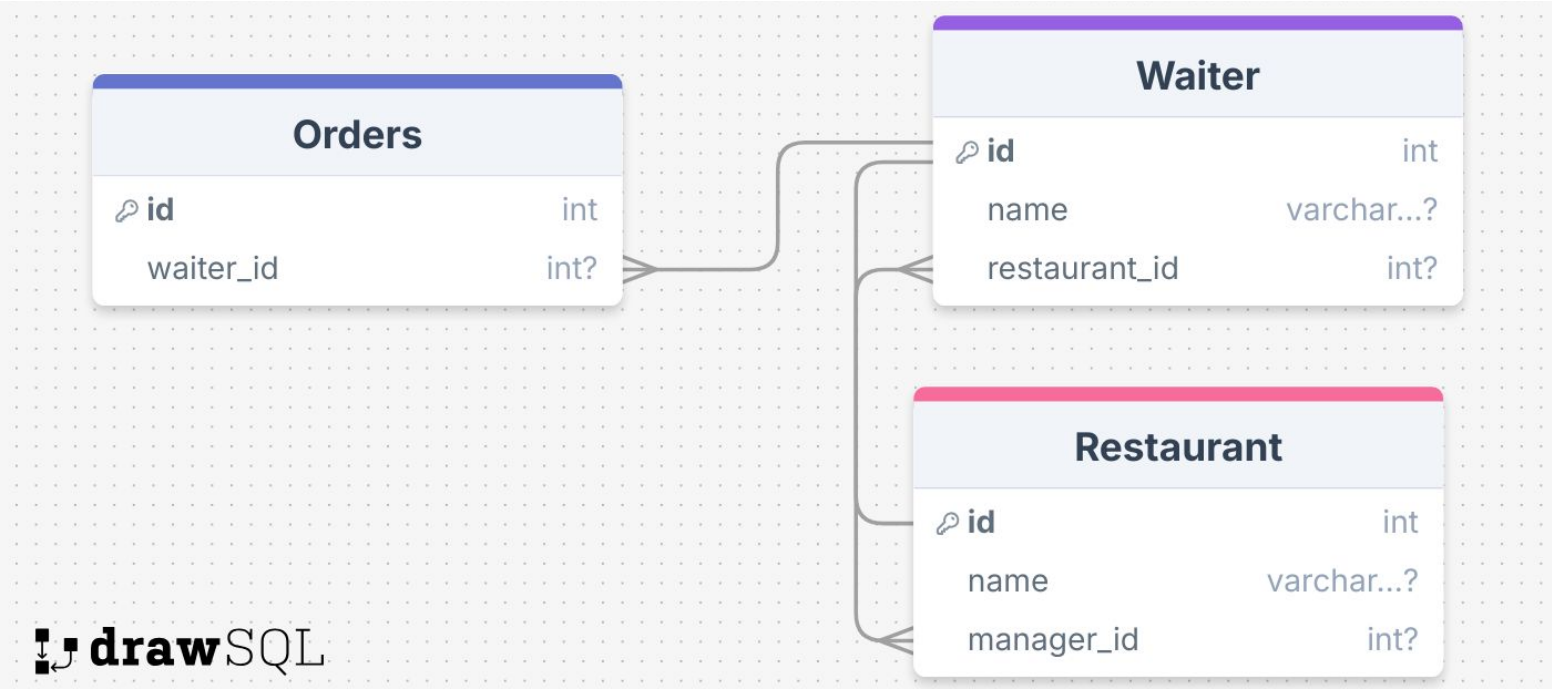
Удаление становится невозможным
каскадные удаления (`ON DELETE CASCADE`) могут зациклиться.

Сложность бизнес-логики
сложно понять, кто главный, а кто подчинённый.

Оптимизатор запросов
может строить неэффективные планы из-за двусмысленных зависимостей.



Цикл возникает между `Restaurant → Waiter → Orders → Restaurant`.
Невозможно создать ресторан без менеджера, менеджера без ресторана, заказ без ресторана и официанта.



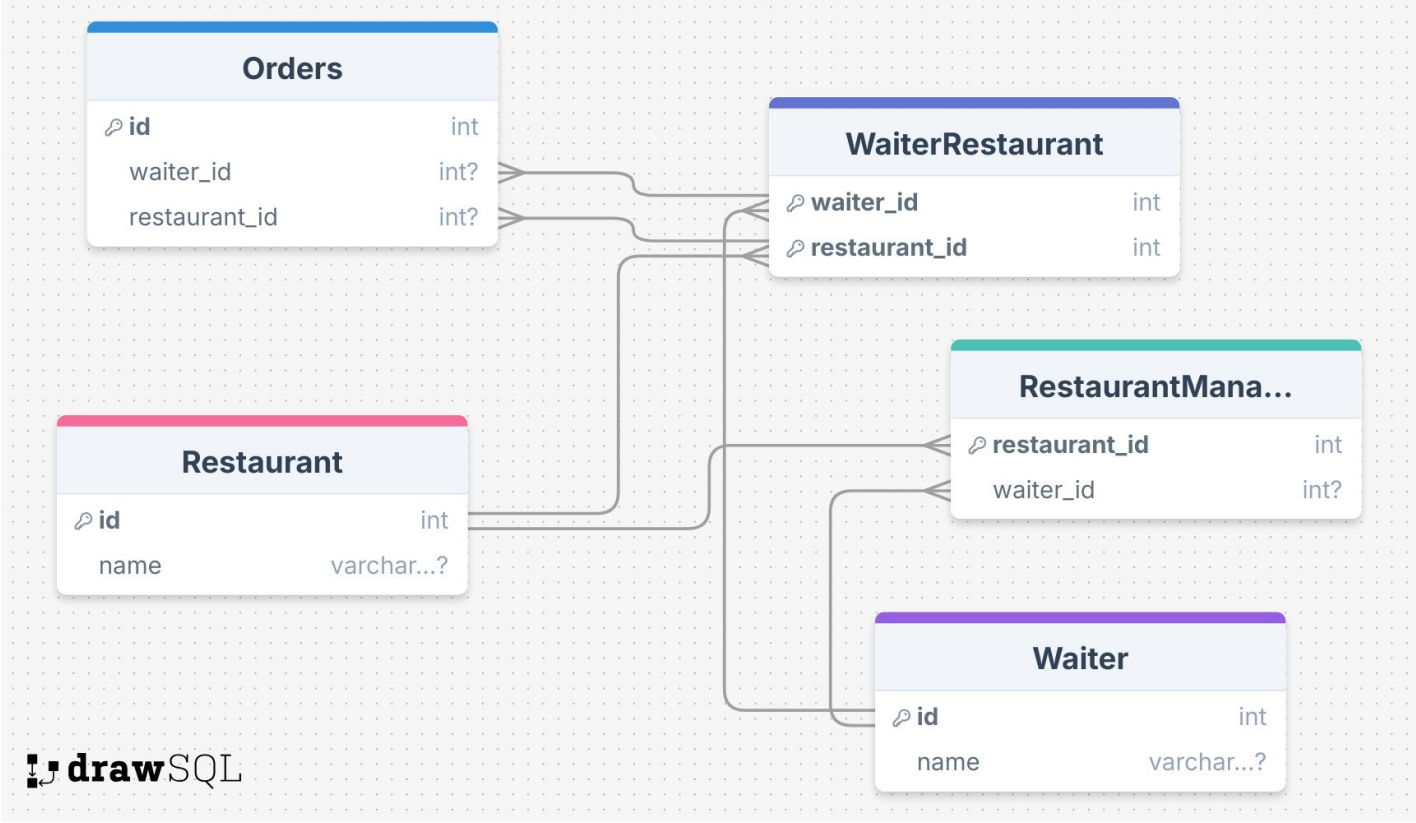
цикл на самом деле остаётся, но в более слабой форме:

- Проблема возникает, если `Restaurant.manager_id` ссылается на официанта, а официант уже привязан к ресторану (`restaurant_id`).
- На этапе вставки данных, чтобы создать ресторан с менеджером, нужен уже существующий официант, а чтобы создать официанта, нужен ресторан.

А что если официант может работать в нескольких ресторанах?

ИТОГ

`WaiterRestaurant` позволяет одному официанту работать в нескольких ресторанах.

`Orders` ссылается на комбинацию `(waiter, restaurant)`, чтобы точно указать место обслуживания.

`RestaurantManager` хранит назначение менеджера отдельно, без создания циклов.